

## 汇编语言指令习题（详解版）

1. 将 1234h 赋给 cx

`Mov cx, 1234h`

这条指令将立即数 1234h 赋值给 cx 寄存器。cx 是一个 16 位寄存器，所以它将存储十六进制数 1234h。

2. 将 bh 的内容设置成和 bl 相同。

`Mov bl, bh`

这条指令将 bh 寄存器的内容复制到 bl 寄存器，使它们相同。这是寄存器之间的数据传递操作。

3. 执行 `mov al, -5h` 后，al 里是什么内容？(二进制表示)

`Mov al, -5h`

这条指令将立即数 -5h 赋值给 al 寄存器。在计算机中，负数通常使用补码表示。 -5h 的二进制表示为 11111011。所以，al 里的内容是 11111011。

4. 将 dl 里的内容存到 1000h 单元。

`Mov [1000h], dl`

解释: 这条指令将 dl 寄存器的内容存储到内存地址 1000h 处。Square brackets [] 表示内存引用。所以，dl 寄存器里的内容将被写入到地址 1000h 的存储单元。

5. 将地址为 1000h 的存储单元里的字节内容拷贝到地址为 2000h 单元。

`Mov dh, [1000h]`

`Mov [2000h], dh`

解释: 第一条指令将从地址 1000h 处读取一个字节的內容，并将其存储到 dh 寄存器中。第二条指令将 dh 寄存器的内容写入到地址 2000h 处的存储单元。这是从一个内存位置复制到另一个内存位置的过程。

6. 将地址为 1000h 的存储单元里的字内容拷贝到地址为 2000h 单元。

`Mov si, [1000h]`

`Mov [2000h], si`

解释: 第一条指令将从地址 1000h 处读取一个字的内容, 并将其存储到 si 寄存器中。第二条指令将 si 寄存器的内容写入到地址 2000h 处的存储单元。这是从一个内存位置复制到另一个内存位置的过程。

7. 将 ds 的值赋给 es。

`Mov ax, ds`

`Mov es, ax`

解释: 第一条指令将 ds 寄存器的值加载到 ax 寄存器中。第二条指令将 ax 寄存器的值存储到 es 寄存器中。这是将数据段寄存器 ds 的值赋给附加段寄存器 es 的过程。

8. 将 1234h 赋给 ds。

`Mov bx, 1234h`

`Mov ds, bx`

解释: 第一条指令将立即数 1234h 存储到 bx 寄存器中。第二条指令将 bx 寄存器的内容赋给数据段寄存器 ds。这是将一个立即数赋给数据段寄存器的过程。

9. 将 66h (字节) 赋给地址为 2000h 的单元。

`Mov byte ptr [2000h], 66h`

解释: 这条指令将立即数 66h 存储到地址 2000h 处的单元中。byte ptr 指定了操作数的大小为字节。所以, 66h 将被写入到地址 2000h 处的存储单元中。

10. 将 66h (字) 赋给地址为 3000h 的单元。

`Mov word ptr [3000h], 66h`

解释: 这条指令将立即数 66h 存储到地址 3000h 处的单元中。word ptr 指定了操作数的大小为字。所以, 66h 将被写入到地址 3000h 处的存储单元中。

## 二. 堆栈指令练习 push pop

1. 将寄存器 ax 的内容入栈。

**Push ax**

这个操作将 **ax** 寄存器的值保存到栈中，栈是一种后进先出（LIFO）的数据结构。

2.将内存地址 1000h 处的内容入栈。

**Push [1000h]**

这个操作将内存地址 1000h 处的内容读取并入栈。

3.将寄存器 **al** 的内容入栈。

**Push ax**

这里可能有误，因为 **al** 寄存器只包含 **ax** 寄存器的低 8 位，而 **Push ax** 操作已经将整个 **ax** 寄存器入栈。这里可能意图是将 **al** 寄存器的值入栈。

4.将常数 1234h 入栈。

**Mov ax,1234h**

**Push ax**

这个操作将常数 1234h 装载到 **ax** 寄存器中，然后将 **ax** 寄存器的值入栈。

5.将栈顶数据存入寄存器 **bx**。

**Pop bx**

这个操作将栈顶的数据弹出并存入寄存器 **bx** 中，即取出上一个 **Push** 操作入栈的数据。

6.将栈顶数据存入内存地址 20h 处。

**Pop [20h]**

这个操作将栈顶的数据弹出并存入内存地址 20h 处。

7.数据交换的三种方法：

A. 使用额外的寄存器 **cx**：

**Mov cx,ax**

**Mov ax,bx**

**Mov bx,cx**

这个方法使用一个额外的寄存器 `cx` 来保存 `ax` 的值，然后通过交换操作实现数据交换。

B. 使用栈操作：

`Push ax`

`Push bx`

`Pop ax`

`Pop bx`

这个方法通过将 `ax` 和 `bx` 的值入栈，然后再出栈，实现数据的交换。

C. 使用 `Xchg` 指令：

`Xchg ax,bx`

`Xchg` 指令直接交换两个寄存器的值，这里用于交换 `ax` 和 `bx` 的值。

8.根据学生编号查询成绩：

`Mov bx, offset mark`；将 `mark` 的地址存入 `bx`

`Mov al, num`；将学生编号存入 `al`

`Xlat`；查表指令，将表中以 `al` 为偏移量的值读取到 `al` 中

这段代码的作用是从存放成绩的表中根据学生编号查询成绩。`Mov bx, offset mark` 将表的起始地址存入 `bx` 寄存器，`Mov al, num` 将学生编号存入 `al` 寄存器，然后使用 `Xlat` 指令查表，将表中以 `al` 为偏移量的值读取到 `al` 中，即查询到了对应学生的成绩。

9.将 `ax` 和内存单元 `y` 相加，结果存于 `y`：

`Add y, ax`

这段代码的作用是将 `ax` 寄存器的值与内存单元 `y` 中的值相加，然后将结果存回内存单元 `y`。

10.将 `ah` 和 `al` 相加，结果存于 `ah`：

`Add ah, al`

这段代码的作用是将 `ah` 寄存器的值与 `al` 寄存器的值相加，然后将结果存回 `ah` 寄存器。

11.将内存单元 x 和内存单元 y 相加，结果存于 x:

`Mov ax, y` ; 将内存单元 y 的值读取到 ax 中

`Mov x, ax` ; 将 ax 的值存入内存单元 x

这段代码的作用是将内存单元 y 中的值加到 ax 寄存器中，然后将 ax 寄存器的值存回内存单元 x。

12.求 bx 减 100:

`Sub bx, 100`

这段代码的作用是将 bx 寄存器的值减去 100，并将结果存回 bx。

13.求 ax、si、进位位之和，结果存于 ax:

`Adc ax, si`

这段代码的作用是将 ax 寄存器的值与 si 寄存器的值相加，同时考虑进位位，结果存回 ax。

14.求 ax 减 si，并减去借位位，结果存于 ax:

`Sbb ax, si`

这段代码的作用是将 ax 寄存器的值减去 si 寄存器的值，同时考虑借位位，结果存回 ax。

15.将内存单元 x 加 1:

`Add x, 1`

这段代码的作用是将内存单元 x 的值加上 1，并将结果存回内存单元 x。

`Inc x`

这段代码也是将内存单元 x 的值加上 1，但是使用 `Inc` 指令，是一个专门的只增指令，直接加 1，不影响进位标志（CF）。两者的功能相同，只是使用的指令不同。

16.将内存单元 y 减 1:

`Sub y, 1`

这段代码的作用是将内存单元 y 的值减去 1，并将结果存回内存单元 y。

`Dec y`

这段代码也是将内存单元 `y` 的值减去 1, 但使用 `Dec` 指令, 是专门的只减指令, 直接减 1, 不影响借位标志 (CF)。两者的功能相同, 只是使用的指令不同。

17. 求 `dx` 的相反数:

`Neg dx`

这段代码的作用是将 `dx` 寄存器的值取相反数, 并将结果存回 `dx`。

18. 比较 `si` 和 `di`:

`Cmp si, di`

这段代码的作用是比较 `si` 寄存器的值和 `di` 寄存器的值, 但不影响寄存器的值, 只设置标志位。

19. 求 `ax` 和 `bx` 的乘积 (有符号数乘法):

`Imul bx`

这段代码的作用是将 `ax` 寄存器的值和 `bx` 寄存器的值相乘, 结果存回 `ax`。这是有符号整数乘法。

20. 设 `x`、`y` 为字节单元, 求 `x * y` (无符号数乘法), 积存于字单元 `z` 中:

`Mov al, x` ; 将 `x` 的值移动到 `al` 寄存器

`Mul y` ; 将 `al` 寄存器的值和 `y` 相乘, 结果存储在 `ax` 寄存器中

`Mov z, ax` ; 将 `ax` 寄存器的值移动到内存单元 `z`

这段代码的作用是将字节单元 `x` 和字节单元 `y` 的无符号数相乘, 结果存储在字节单元 `z` 中。首先, 将 `x` 移动到 `al` 寄存器, 然后执行无符号数乘法指令 `Mul y`, 结果存储在 `ax` 寄存器中, 最后将 `ax` 寄存器的值移动到内存单元 `z`。

21. 将 `al` 扩展到 `ax`:

`Cbw`

这段代码的作用是将 `al` 寄存器的值扩展到 `ax` 寄存器。`Cbw` 指令 (Convert Byte to Word) 执行有符号扩展, 将 `al` 寄存器的最高位 (符号位) 复制到 `ah` 寄存器, 实现有符号数的扩展。

22.将 bh 扩展到 bx:

Mov al, bh ; 将 bh 寄存器的值移动到 al 寄存器

Cbw ; 将 al 寄存器的值扩展到 ax 寄存器

Mov bx, ax ; 将 ax 寄存器的值移动到 bx 寄存器

这段代码的作用是将 bh 寄存器的值扩展到 bx 寄存器。首先，将 bh 的值移动到 al 寄存器，然后使用 Cbw 指令将 al 寄存器的值扩展到 ax 寄存器，最后将 ax 寄存器的值移动到 bx 寄存器。

23.将 ax 扩展到 dx:ax:

Cwd

这段代码的作用是将 ax 寄存器的值扩展到 dx:ax 寄存器对，其中 dx 寄存器接收 ax 的符号位（最高位）。Cwd 指令（Convert Word to Doubleword）执行有符号扩展，将 ax 寄存器的符号位复制到 dx 寄存器。

24.逻辑运算指令练习

已知有内存变量:

X1=12h,x2=34h,x3=56h,x4=78h,x5=9ah,x6=0bch,x7=0deh,

y1=12h,y2=34h,y3=56h,y4=78h,y5=9ah,y6=0bch,y7=0deh,另设 (al) =0abh,求:

求 x1 与 al

求 x2 或 al

求 x3 异或 al

求 x4 位相反数

将 x5 的高四位清零

将 x6 的低三位置 1

将 x7 的第 0、2、4、6 位取反

测试 al 是否为偶数?

将 y1 逻辑左移 2 位

将 y2 逻辑右移 3 位

将 y3 算术左移 1 位

将 y4 算术右移 4 位

将 y5 循环右移 2 位

将 y6 循环左移 1 位

将 y7 带进位循环右移 1

解: data segment

X1 db 12h

X2 db 34h

X3 db 56h

X4 db 37h

~

~

Y7 db 0deh

Data ends

Code segment

Assume cs:code,ds:data

Start: mov ax,data

Mov ds,ax

Mov al,0abh

And x1, al (求 x1 和 al)

Or x2, al (求 x2 或 al)

Xor x3, al (求 x3 异或 al)

Not x4 (x4 的相反数)

And x5,0fh (将 x5 的高四位清空)

Or x6,00000111b (将 x3 的低三位置 1)

Xor x7,01010101b (将 x7 的 0,2,4,6,8 位相反)

Test al,1 (测试 al 是否为偶数)

Shl y1,2 (逻辑左移 2 位)

Shr y2,3 (逻辑右移 3 位)

Sal y3,1 (算术左移 1 位)

Sar y4,4 (算术右移 4 位)



Ror y5,2 (循环右移 2 位)

Rol y6,1 (循环左移 1 位)

Rcr y7,1 (将 y7 带进位循环右移 1 位)

三.

1.按要求完成进制转换

(1)78ch=011110111100 b=1980 d

(2)562q= 101110010b=370d

(3)90d= 01011010b=5a h

(4)1110100.111b= 164.7q=74.eh

2.当字长位八位时，写出下面各数的原码，补码和反码

(1) 25

(2) -25

(3) 100

(4) -100

原码:00011001      10011001      01100100      11100100

反码:00011001      11100110      01100100      10011011

补码:00011001      11100111      01100100      10011100

(真值的由来  $100=64+32+4=26+25+22=0*27+1*26+1*25+0*24+0*23+1*22+0*21+0*20$ ,

所以 100 的真值是 01100100,正数的真值=原码=补码=反码)

3.列出数字 0~9、大写字母 a~z、小写字母 a~z 的 ascii 码值

30h~39h

41h~5ah

61h~7ah

4.完成下列二进制数的加减运算

(1) 10101010+11110000=110011010

(2) 11001100+01010100=100100000

(3) 11011010-01010010=10001000

(4) 11101110-01001101=10100001

5.说明二进制(bit)、字节(byte)、字(word)、双字(double word)的含义及互相的关系。

位(bit):一个二进制

字节 (byte) :8 位二进制

字 (word) :一般为机器字长,在 8086 机器中为 16 位二进制,即两个字节

双字 (double word) :双倍字长,在 8086 机器中为 32 位二进制

6.8088 微处理器中有哪些寄存器? 说明它们的功能及分组情况?

(1) 通用寄存器组 (数据寄存器):

AX (累加器寄存器): 用于累加运算和存储结果。

BX (基址寄存器): 通常用作数据的基址。

CX (计数寄存器): 用于循环计数和移位操作。

DX (数据寄存器): 通常用于存放 I/O 端口地址。

(2) 指针寄存器组:

SI (源变址寄存器): 用于源操作数的变址。

DI (目的变址寄存器): 用于目的操作数的变址。

(3) 基址寄存器组:

BP (基址指针寄存器): 通常用于指向堆栈中的数据。

SP (堆栈指针寄存器): 指向堆栈顶部。

(4) 段寄存器组:

CS (代码段寄存器): 存储代码段的起始地址。

DS (数据段寄存器): 存储数据段的起始地址。

ES (附加段寄存器): 通常用于附加数据段的起始地址。

SS (堆栈段寄存器): 存储堆栈段的起始地址。

(5) 标志寄存器:

FLAGS: 包含各种标志位,如零标志、进位标志、溢出标志等。

7.一台微型计算机,数据线有 8 根,地址线有 16 根,如果采用字节编址,那么它可以访问的最大存储空间是多少字节? 试用十六进制数来表示该机的地址范围?

答:  $2^{16}=64\text{kb}$ , 0000h~ffffh

8.8086 的寄存器中,有哪些 16 位寄存器可分为两个 8 位寄存器来使用?

答: ax (ah,al)、bx (bh,bl)、cx (ch,cl)、dx (dh,dl)

9.8086/8088 的标志寄存器 f 种有哪些标志位？它们的名称和功能是什么？

答：cf—进位标志 (carry flag)。若 cf=1，表示算术运算时产生进位或借位，否则 cf=0。

移位指令会影响 cf

pf—奇偶标志 (parity flag)。若 pf=1，表示操作结果中“1”的个数为偶数，否则

pf=0。这个标志主要用于检查数据传送过程中的错误。

af—辅助进位标志 (auxiliary carry flag)。若 af=1 表示字节运算产生低半字节向高半字节的进位或错位，否则 af=0。辅助进位也称半进位标志，主要用于 bcd 码运算的十进制调整。

zf—全零标志 (zero flag)。若 zf=1，表示操作结果全为零，否则 zf=0。

sf—符号标志 (sign flag)。若 sf=1，表示符号数运算后的结果为负数，否则 sf=0。

of—溢出标志 (overflow flag)。若 of=1，表示当进行算术运算时，结果超过了最大范围，否则 of=0。

if—中断允许标志 (interrupt flag)。若 if=1，则 cpu 可以响应外部可屏蔽中断请求；若 if=0，则 cpu 不允许响应中断请求。If 的状态可由中断指令设置。

df—方向标志 (direction flag)。若 df=1，表示执行字符串操作时按着从高地址向低地址方向进行；否则 df=0。df 位可由指令控制。

tf—单步标志 (trace flag)。又称跟踪标志。该标志位在调试程序时可直接控制 cpu 的工作状态。当 tf=1 时为单步操作，cpu 每执行一条指令就进入内部的单步中断处理，以便对指令的执行情况进行检查；若 tf=0，则 cpu 继续执行程序。

10.在 8086 系统中，段地址和偏移地址如下表示，请分别计算它们的物理地址

- |               |                          |
|---------------|--------------------------|
| (1) f000:0100 | 物理地址：f0000h+0100h=f0100h |
| (2) 2350:1abc | 23500h+1abch=24fbc       |
| (3) 3200:1234 | 32000h+1234h=33234h      |
| (4) 5000:010f | 50000h+010eh=5010eh      |
| (5) 10e0:2020 | 10e00h+2020h=12e20h      |
| (6) 2010:1020 | 20100h+1020h=21120h      |
| (7) 1000:1000 | 10000h+1000h=11000h      |
| (8) c0c0:c0c0 | c0c00h+c0c0h=cccc0h      |

11.8086/8088 指令系统中操作数的类型有哪几种？关于操作数的寻址方式有几类？

答：（1）立即数、寄存器数和内存单元数。

（2）立即寻址、寄存器寻址、寄存器间接寻址、直接寻址、寄存器相对寻址、基址加变址寻址、基址加变址相对寻址

12.指出段地址、偏移地址与物理地址之间的关系。有效地址 ea 是指什么？

答：段地址左移四位加上偏移地址形成 20 位的物理地址。

ea 是指段内偏移地址，即表示段内某单元相对于段起始地址的空间位置。

13.指出能用于寄存器间接寻址及变址寻址的寄存器有哪些？它们通常与哪个段寄存器配合形成物理地址？

答：能用于寄存器间接寻址及变址寻址的寄存器有基址寄存器 bx 和 bp，变址寄存器 si 和 di, bx、si、di 与 ds 配合形成物理地址，而 bp 与 ss 配合形成物理地址。

14.用汇编语言指令实现以下操作

（1）将寄存器 ax、bx 和 dx 的内容相加，和放在寄存器 dx 中

```
add dx,ax
```

```
add dx,bx
```

（2）用基址变址寻址方式（bx 和 si）实现 al 寄存器的内容和存储器单元 buf 中的一个字节相加的操作，和放到 al 中。

```
add al,[bx][si]
```

（3）用寄存器 bx 实现寄存器相对寻址方式（位移量为 100h），将 dx 的内容和存储单元中的一个字相加，和放到存储单元中。

```
add 100h[bx],dx
```

（4）用直接寻址方式（地址为 0500h）实现将存储器中的一个字与立即数 3abch 相加，和放回该存储单元中

```
add word ptr[500h],3abch
```

15.指出下列指令中源操作数及目的操作数的寻址方式。

（1）sub bx,[bp+35]

寄存器寻址      寄存器相对寻址

(2) mov ax,2030h

寄存器寻址      立即寻址

(3) mov [di+bx],ax

基址加变址寻址      寄存器寻址

(4) add ax,50h[di]

寄存器寻址      寄存器相对寻址

(5) mov al,[1300h]

寄存器寻址      直接寻址

(6) mul bl

寄存器寻址      目的操作数为隐含寄存器（bl 隐含的是 al，若 bx，则隐含 ax）

16. 已知 (ds) = 1000h, (si) = 0200h, (bx) = 0100h, (10100h) = 11h, (10101h) = 22h,

(10600h) = 33h, (10601h) = 44h, (10300h) = 55h, (10301h) = 66h, (10302h) = 77h,

(10303h) = 88h, 试分析下列各指令执行完后 ax 寄存器的内容。

(1) Mov ax,2500h

(ax) = 2500h

(2) Mov ax,500h[bx]

(ax) = 4433h

(3) Mov ax,[300h]

(ax) = 6655h

(4) Mov ax,[bx]

(ax) = 2211h

(5) Mov ax,[bx][si]

(ax) = 6655h

(6) Mov ax,[bx+si+2]

(ax) = 8877h

17. 判断下列指令是否有错，如果有错，说明理由

(1) sub bl,bx

两个操作数的宽度不一样

(2) `mov byte ptr[bx],3456h`

3456h 太大

(3) `shl ax,ch`

移位的位数不能是 ch (只能是 cl)

(4) `mov ah,[si][di]`

不能用两个变址寄存器来作为源操作数

(1.[si]、[di]2.[bx]、[bp]只能 1 和 2 配, 不能 1 和 1 或是 2 和 2 配)

(5) `shr ax,4`

对的

(6) `mov cs,bx`

不能对 cs 实现传送操作

(7) `mov 125, cl`

立即数不能作为目的操作数 (若是 125 加上 [] 变为内存, 则行)

(8) `mov ax,byte ptr[si]`

字节不能传送到字里面 (源操作数限定为字节, 与目的操作数宽度不同, 不能实现传送)

(9) `mov [di],[si]`

两个操作数不能同时为内存

12. 设 (ds) = 1000h, (es) = 2000h, (ss) = 3000h, (si) = 0080h, (bx) = 02d0h, (bp) = 0060h,

试指出下列指令的源操作数字段是什么寻址方式, 它的物理地址是多少?

(1) `mov ax,0cbh`

立即寻址

(2) `mov ax,[100h]`

直接寻址

物理地址为: 10100h

(3) `mov ax,[bx]`

寄存器间接寻址

物理地址为: 102d0h

(4) `mov ax,[bp]`

寄存器间接寻址

物理地址为: 20060h

(5) `mov ax,[bp+50]`

寄存器相对寻址

物理地址为: 200b0h

(6) `mov ax,[bx][si]`

基址加变址寻址

物理地址为: 10350h

13.分别说明下列每组指令中的两条指令的区别

(1) `and cl,0fh`                      `or cl,0fh`

按位相“与”

按位相“或”

高四位为 0000

高四位保留原值

低四位保留原值

低四位为 1111

(2) `mov ax,bx`                                      `mov ax,[bx]`

bx 为寄存器的内容送到 ax

bx 为寄存器内存单元的内容送到 ax

(3) `sub bx,cx`                      `cmp bx,cx`

把相减的结果给 bx

不把结果给 bx, 根据标志位的情况做进一步的动作

(4) `and al,01h`                                      `test al,01h`

Al 内容与 01h 相“与”

•

结果为“000000x”

•

把结果送回 al

不把结果给 al (即不保存结果)

(5) `rol ax,cl`                                      `rcl ax,cl`

循环左移, 进位标志不参与循环

循环左移, 进位标志参与循环

(6) `push ax`                                      `pop ax`

入栈 (将 ax 内容存入栈顶指针处)

出栈 (将栈顶内容弹出装入 ax 中)

14.试分析以下程序段执行完后 bx 的内容是什么

`mov bx,1030h`

1030h=1000000110000

|                        |                                 |
|------------------------|---------------------------------|
| <code>mov cl,3</code>  | 左移 3 位得 1000000110000000=8180h  |
| <code>shl bx,cl</code> |                                 |
| <code>dec bx</code>    | dec 减 1 得 1000000101111111=817f |

15. 写出下列指令序列中每条指令的执行结果,并在 debug 环境下验证,注意各标志位的变化情况.

```

mov bx,126bh
add bl,02ah
mov ax,2ea5h
add bh,al
sbb bx,ax
adc ax,26h
sub bh,-8

```

16.8086 汇编语言指令由几部分组成?各部分的作用是什么?

答: (1) 由[名字](指令操作符)[操作数][注释]组成

(2) [名字]可以是常量名, 变量名, 过程名, 段名等标识符, 起标识和标记的作用。

(指令操作符) 字段说明指令的功能

[操作数]字段提供指令操作所需要的源数据和目的数据

[注释]用于程序员对语句或程序段落进行解释说明

17. 请解释变量和标号的含义,两者有何区别?

答: 所定义内存单元的名称为变量, 可进行读写功操作。一般为数据区。

标号是某条指令的地址标记, 在代码段

18. 试描述汇编语言源程序的一般结构

```

stack segment stack
        dw 256 dup ( ? )
stack ends

```



```

data segment
.....
data ends

code segment

assume cs: code,ss: stack,ds: data

start: .....
.....

code ends

end start

```

19.在 buf1 变量中一次存储了 5 个字数据,接着定义了一个名为 buf2 的字单元,表示如下

```

buf1 dw 8765h,6ch,0,1ab5h,47eah
buf2 dw ?

```

(1)设 bx 中是 buf1 的首地址,请编写指令将数据 50h 传送给 buf2 单元

```
mov mword ptr[bx+10],50h
```

(2)请编写指令将数据 ffh 传送给数据为 0 的单元.

```
mov mword ptr[bx+4],0ffh
```

20.下面是一个数据段的定义,请用图表示它们在内存中存放的形式

```

data segment

a1 db 25h,35h,45h

a2 db 3 dup(5)

a3 dw 200,3ab6h

a4 dw 3000h,6a6fh

data ends

```

答: a1 25h

35h

45h

a2 05h

05h  
05h  
a3 00h  
02h  
b6h  
3ah  
a4 00h  
30h  
6fh  
6ah

21.有一个数据段定义了如下 6 个变量,请写出该数据段

- (1)buf1 为十进制数字节变量:64;
- (2)buf2 为字符串变量:'teacher';
- (3)buf3 为十六进制数字节变量: 2fh;
- (4)buf4 为双字变量: 657ah;
- (5)buf5 为字变量: 657ah;
- (6)buf6 为二进制数字节变量: 10101101b。

```
data segment  
buf1 db 64  
buf2 db 'teacher'  
buf3 db 2fh  
buf4 dd 657ah  
buf5 dw 657ah  
buf6 db 10101101b
```

一、熟悉用汇编语言编写程序的基本结构

现有程序:

Data segment

first dw f234h,1122h(不能以字母开头,在前面加零,写成 0f234h)

second dw 5678h, 9765h(要在英文状态下写,才是半角)

third dw ?,?

Data ends

Code segment

assume cs:code,ds:data(是 assume)

Start: mov ax,data(是 o,不是零)

mov ds,ax

mov ax,first1(bu yao 1)

add ax,second

mov third,ax

mov ax,first+2

adc ax,second+2

mov third+2,ax

jmp \$

Code ends

ends start(改为 end)

- 1、在系统中输入上述程序,编译后改正错误,直到调试通过。(提示:有6处错误)
- 2、为每个语句加上注释,写清每个语句的含义和作用。
- 3、整个程序完成什么功能?在系统中查看运行后的结果。(48ach,0410h)

三、将下列注释中的问题用指令编程实现。

Data segment

;定义三个变量: x1 =12h,      x2 = 34h,      x3= 56h

Data ends

Code segment

assume cs:code,ds:data

Start: ;将 data 段初值装载到 ds 段寄存器中

mov ax,1111h

mov bx,2222h

mov dx,3333h

mov cx,0cccch

;1、将寄存器 ax、bx 和 dx 内容相加，和放在寄存器 dx 中，ax、bx 中值不变。

;2、将 x1 的值 送给 y

;3、将 x2 的值减去 al 的值。

;4、求 x3 的相反数。

;5、将 cx 的最高位清 0.

;6、将 cx 的最低 3 位置 1.

;7、将 cx 的第 13、14 位取反。(最低位为第 0 位)

;8、将 ax 逻辑左移 2 位。

;9、将 bx 算术右移 1 位。

;10、将 bx 循环左移 3 位。

jmp \$

code ends

end start

解: data segment

X1 db 12h

```

X2 db 34h

X3 db 56h

Y db ?

Data ends

Code segment

Assume cs:code,ds:data

Start:mov ax,data

      mov ds,ax

      mov ax,1111h

      mov bx,2222h

      mov dx,0ccch

      add dx,ax

      add dx,bx

      mov al,x1

      mov y,al

      sub x2,al

      neg x3

      and cx,01111111b

      or cx,00000111b

      xor cx,0000000000001100b

      shl ax,2

      sar bx,1

      rol bx,3

      jmp $

code ends

end start

```

#### 四、顺序程序设计

- 1、计算  $z=x-y$ 。其中， $x$ 、 $y$ 、 $z$  均为字节型变量。设  $x=4$ ， $y=6$ 。要求：运算后， $x$ 、 $y$  的值不被破坏。检查  $z$  中的结果显示什么？其真值是多少？

```
data segment
```

```

X    db, 4
Y    db, 6
Z    db, ?

Data ends

Code segment

    assume cs:code, ds:data

start:mov ax, data

        mov dx, ax

        mov ax, 4

        mov bx, 6

        sub ax, bx

        mov dx, ax

        mov z, dx

        jmp $

    code ends

end start

```

2、编程将字型变量 a 的高 4 位置 0，低 4 位置 1，中间 8 位取反。

解：

```

data segment

A    dw 1234h

Data ends

Code segment

Assume cs:code, ds:data

Start:mov ax, data

    mov ds, ax

    mov ax, a

    and ax, 0fffh

    or ax, 0fh

    xor 0ff0h

    mov a, ax

    jmp $

```

Code ends

End start

4、设内存字单元 x 中存有 0 至 9 中任一自然数 x，求其立方值，存于字单元 y。

Data segment

X dw, 8

Y dw, ?

Tab dw, 0, 1, 8, 27, 64, 125, 216, 343, 512, 729

Data ends

Code segment

assume cs:code, ds:data

start:

mov ax, data

mov dx, ax

mov bx, x

shl bx, 1

mov ax, tab[bx]

mov y, ax

jmp \$

code ends

end start

## 五、分支程序设计

1、计算  $y=\text{sgn}(x)$  (当  $x>0$  时,  $y=1$ ;  $x<0$  时,  $y=-1$ ;  $x=0$  时,  $y=0$ )

```
data segment
X db, 1h
Y db, ?
Data ends

Code segment
Assume cs:code, ds:data

Start:

    mov ax, data
    mov ds, ax
    mov ax, 1
    cmp ax, 0
    ja above
    jz zero
    less:
    mov y, -1
    jmp done
    above:
    mov y, 1
    jmp done
    zero:
    mov y, 0
    jmp done
done: jmp $
code ends
end start
```

2、试编写一个程序, 求 3 个无符号字数数据中的最大值, 并将最大值存入 max 字单元中。

解: data segment



```

X db 12
Y db 44
Z db 77
Max db ?
Data ends

Code segment

Assume cs:code,ds:data

Start:mov ax,data

      mov ds,ax

      mov al,x

      cmp al,y

      Jae flag1(取的名字)

      mov al,y

Flag1:cmp al,z

      Jae flag2

      Mov al,z

Flag2:mov max,al

      Jmp $

Code ends

End start

```

## 六、循环程序设计

1、实现自然数 1~100 的累加，结果存放在字变量 result 中。

```

Data segment

Result dw ?

Data ends

Code segment

      assume cs:code,ds:data

      start:mov ax,data

            mov ds,ax

```

```

mov cx, 100

mov ax, 0

mov bx, 1

l1: add ax, bx

    inc bx

    loop l1

mov result, ax

jmp $

code ends

end start

```

2、已知 buf 开始的数据区中存放有 10 个英文字母，试编写一个程序将其中的小写字母全部转换成大写字母。

Data segment

Buf db "abcdeef123"

Data ends

Code segment

```

assume cs:code, ds:data

start: mov ax, data

    mov ds, ax

    mov cx, 10

    mov bx, offset buf

flag0: mov al, [bx]

    cmp al, 'a'

    jb flag1

    cmp al, 'z'

    sub al, 20h

    mov [bx], al

flag1: inc bx

    loop flag0

```

```

    jmp $

code ends

end start

```

3、在字符串 string 中查找字符 a，若找到，在 result 变量中存入“y”，若没找到，存入“n”。

解：data segment

```

    N            dw    ?

    Y            dw    ?

    Result       dw    ?

    String       db    "this is a string for example. ", 0

Data ends

```

Code segment

Assume ds: data, cs: code

Start:

```

mov  ax, data
mov  ds, ax
mov  si, 0

```

L0: cmp string[si], 0

jz notfound

cmp string[si], 'a'

jz found

inc si

jmp l0

Found: mov ax, y

mov result, ax

jmp exit

Notfound: mov cx, n

mov result, cx

Exit: jmp \$

Code ends

end start

4、字符串 string 以代码 0 结束，求这个字符串的长度。

```
解: data segment
    String db "a.",0
    Lenth dw ?
Data ends

Code segment
Assume cs:code,ds:data

Start:mov ax,data

        Mov ds,ax

        Mov si,offset string

        Mov cx,0

Tst:cmp byte ptr[si],0

        Je done

        Inc si

        Inc cx

        Jmp tst

Done:mov lenth,cx

        Jmp $

Code ends

End start
```

## 七、子程序设计

1、在缓冲区 marklist 中存放了 10 名同学的成绩，将其转换为相应的等级成绩，存入缓冲区 gradelist 中。转换过程采用子程序实现。

转换规则为：

>=90: a

>=80 且 <90: b

>=60 且 <80: c

<60: d

不用考虑边界情况，即假设输入的分数成绩都在 0-100 分中间。

数据段定义如下：

Data segment

```
marklist db 92,82,72,52,90,80,60,30,99,88
```

```
gradelist db 10 dup(?)
```

Data ends

子程序接口定义如下：

子程序名：change

输入参数：ah，分数成绩

输出参数：al，等级

解：data segment

```
Marklist db 92,82,72,52,90,80,60,30,99,88
```

```
gradelist db 10 dup(?)
```

```
data ends
```

```
code segment
```

```
assume cs:code,ds:data
```

```
start proc
```

```
mov ax,data
```

```
mov ds,ax
```

```
mov cx,10
```

```
mov si,offset marklist
```

```
mov bx,offset gradelist
```

```
l1:mov ah,[si]
```

```
call change
```

```
mov [bx],al
```

```
inc si
```

```
inc bx
```

```
loop l1
```

```
        jmp $
        ret
start   endp
change  proc
        cmp  ah,90
        jb   m1
        mov  al,'a'
        jmp  exit
m1:cmp  ah,80
        jb   m2
        mov  al,'b'
        jmp  exit
m2:cmp  ah,60
        jb   m3
        mov  al,'c'
        jmp  exit
m3:mov  al,'d'
        exit:ret
change  endp
code    ends
end      start
```

本文件部分内容由**公众号：数基培文**整理，数学/计算机/文学内容，欢迎关注，获取更多大学学习资料。